

Lab overview

This lab provides me with hands-on experience using AWS CloudFormation, Amazon Elastic Compute Cloud (Amazon EC2) and the AWS Command Line Interface (CLI). I'll review an incomplete CloudFormation template and be challenged to fill in the missing parameters, allowing the stack to launch an Amazon EC2 instance, and user data install and run nginx. Once the stack is created, I'll verify that nginx is running on the Amazon EC2 instance.

Objectives

By the end of this lab, I should be able to do the following:

- Review an incomplete CloudFormation template.
- Update the CloudFormation template to create an Amazon EC2 instance.
- Configure the instance to use user data to install nginx.
- Create a CloudFormation stack with an Amazon EC2 resource.
- Test that the nginx web server runs successfully on the Amazon EC2 instance.

Task 1: Review an incomplete CloudFormation template that creates an Amazon EC2 instance

In this task, I'll connect to a VS Code that is used as the Integrated Development Environment (IDE) to review and update the incomplete CloudFormation template that has been prebuilt for me. I'll identify the properties that have been defined and determine which parameters need to be updated for the template to successfully create a new CloudFormation stack and the resource(s) defined.

```
1 AWSTemplateFormatVersion: 2010-09-09
2 Description: >
3 This template creates an Amazon EC2 instance. It installs and configures
4 the Nginx web server via user data.
5
6 Parameters:
7   AmazonLinuxAMIID:
8     Type: AWS::S3::Parameter::ValueFromS3::Image::Id
9     Default: /aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64
10
11 Resources:
12   WebServer:
13     Type: AWS::EC2::Instance
14     Properties:
15       ImageId: !Ref AmazonLinuxAMIID
16       InstanceType: t3.micro
17       SecurityGroupIds:
18         - ???
19       SubnetId: ???
20       UserData:
21         Fn::Base64: !Sub [
22           #!/bin/bash
23           yum install -y nginx
24           systemctl enable nginx
25           systemctl start nginx
26
27 Outputs:
28   Website:
29     Description: URL to site
30     Value: !Sub 'http://${WebServer.???}'
```

This CloudFormation template provides a blueprint to create an Amazon EC2 instance and installs the nginx web server on it. Here is the detailed overview of each section:

Detailed overview

- ***AWSTemplateFormatVersion:***
 - Specifies the AWS CloudFormation template version that the template conforms to.
- ***Description:***
 - Provides a text description of the template. It describes the purpose of the template, which is to create an EC2 instance and install the nginx web server.
- ***Parameters:***
 - Allow for the input of custom values during stack creation or updates. In this template, there's one parameter defined:
 - **AmazonLinuxAMIID:** This specifies the Amazon Machine Image (AMI) ID that the Amazon EC2 instance uses. This is fetched from the Systems Manager Parameter Store's predefined AMI ID for Amazon Linux.
- ***Resources:***
 - Define the AWS resources that are created or modified. In this case, there is one resource defined:
 - **WebServer:** An Amazon EC2 instance. It has the following properties:
 - **ImageId:** The ID of the AMI used to launch the instance. (This value is missing and needs to be filled in with a parameter by me as part of the challenge for this lab.)
 - **InstanceType:** The instance type (size) which determines its CPU, memory, storage, and network capacities.
 - **SecurityGroupIds:** Specifies the security group(s) associated with the instance. (This value is missing and needs to be filled in with a security group resource or an existing security group's ID by me as part of the challenge for this lab.)
 - **SubnetId:** Specifies the ID of the subnet in which to launch the instance. (This value is missing and needs to be filled in with a subnet's ID from an existing VPC by me as part of the challenge for this lab.)
 - **UserData:** Contains a script that is run when the instance starts. It installs and starts the nginx web server.

UserData: detailed overview

The given user data is a bash script that is run automatically when an Amazon EC2 instance is launched. Below the specific commands are broken down to explain their purpose:

- ***#!/bin/bash:*** This is called a *shebang* and indicates that the script should be interpreted and run using the bash shell.
- ***yum install -y nginx*** This command uses the *yum* package manager (specific to certain Linux distributions including Amazon Linux) to install the nginx web server. The *-y* flag is used to automatically answer yes to any prompts during the installation.

- ***systemctl enable nginx***: This command allows the nginx service to start automatically upon system boot. So, whenever the Amazon EC2 instance is restarted, nginx is started as well.
- ***systemctl start nginx***: This command starts the nginx service immediately after it's installed, making the web server active and ready to serve web pages.
- **Outputs:**
 - Return values or data that can be imported into other stacks or queried using the AWS Management Console, AWS CLI, and AWS SDKs. This section has:
 - **Website**: Outputs the URL where the nginx website can be accessed. However it appears to be missing the full reference to get the public IP of the *WebServer* instance. (This value is missing and needs to be filled in with the missing details by me as part of the challenge for this lab.)

To make this CloudFormation template complete and functional, the missing values (represented by ???) need to be filled in.

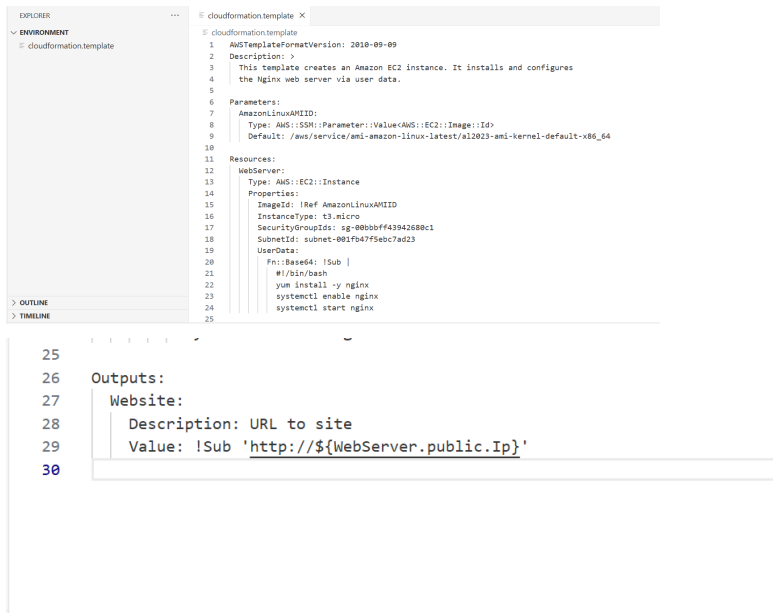
I have reviewed the incomplete CloudFormation template and identified the properties that require updating for the template to successfully create a new stack with an Amazon EC2 resource.

Task 2: Update the incomplete CloudFormation template

In this task, I am challenged with updating the incomplete CloudFormation template with the missing details so that it can create a new CloudFormation stack and any defined resource(s). The resources section of the template contains three properties that need to be updated. These properties are specified by ??? (three question marks). There is one last update required for this CloudFormation template to be complete and functional. The output section is missing details to provide the public IP address of the instance when processed. I can retrieve the public IP address of an Amazon EC2 instance by accessing the instance's *return values*. This is a list of attributes that can be returned when I use either the `!GetAtt` or `!Sub` intrinsic function.

- `!GetAtt` is specifically designed to retrieve attributes from CloudFormation resources.
- `!Sub` is more versatile and can be used for string substitutions, including embedding `!GetAtt` functions within it to replace placeholders with actual resource attributes.

As you can see, in my CloudFormation template, it is using the `!Sub` intrinsic function because it is creating a string for a proper URL address. When the output is processed, it provides the full URL of the instance's public IP address as `http://1.1.1.1` (where 1.1.1.1 is updated with the actual public IP address).



I have filled in the missing property values for the Amazon EC2 resource in the incomplete CloudFormation template. I also added the missing Amazon EC2 PublicIp attribute required to retrieve the Amazon EC2 instance's public IP address.

Task 3: Create a CloudFormation stack and verify the user data processed successfully

In this task, I'll create a CloudFormation stack using my CloudFormation template which creates an Amazon EC2 instance. Once the stack finishes, I'll run an AWS CLI command to retrieve the *Output* to see the nginx URL that gets created as part of the stack creation process. Then I'll copy and paste the URL into a browser to verify the nginx server is functional and responding to web requests.

Task 3.1: Create the CloudFormation stack using the AWS CLI

Using the AWS CLI, I can create a CloudFormation stack and the associated resource(s) specified in the CloudFormation template file. To launch a CloudFormation stack using a template file via the AWS CLI, I'll use the *aws cloudformation create-stack* command.

I'll create the CloudFormation stack using the following AWS CLI command in the VS Code IDE terminal.

```
aws cloudformation create-stack --stack-name nginx-Web-Server --template-body file://~/environment/cloudformation.template
```

I'll Check the status of the CloudFormation stack creation with the following command.

```
aws cloudformation describe-stacks --stack-name "nginx-Web-Server" --query "Stacks[0].StackStatus"
```

To retrieve the URL for the nginx web server, I'll run the following command.

```
aws cloudformation describe-stacks --stack-name "nginx-Web-Server" --query "Stacks[0].Outputs"
```

Task 3.2: Verify the user data processed successfully by testing the nginx web server

I can now open the link to the public IP address of the Amazon EC2 instance to confirm the user data script ran successfully and that the nginx web server is running and accepting web requests.

I should see a welcome message as show below:

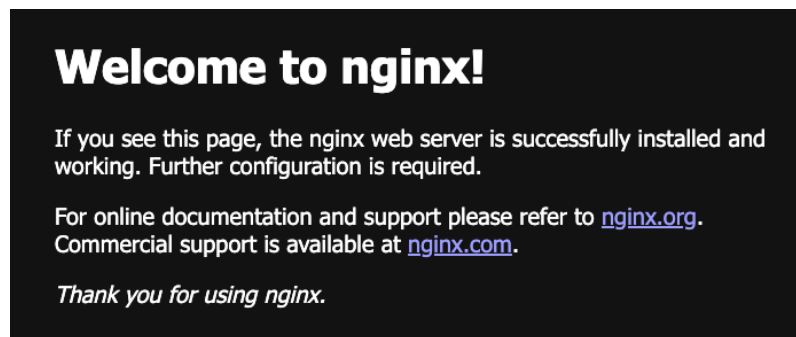


Image description: The preceding diagram depicts a sample website page with the message "Welcome to nginx!".

This indicates that the user data ran and completed successfully when I created the instance using CloudFormation.

I have created a CloudFormation stack using the CloudFormation template file that I updated. I retrieved the nginx URL and verified that the user data was processed successfully by viewing the nginx default home page running locally on the Amazon EC2 instance I created using CloudFormation.

Conclusion

I successfully did the following:

- Reviewed an incomplete CloudFormation template.
- Updated the CloudFormation template to create an Amazon EC2 instance.
- Configured the instance to use user data to install nginx.
- Created a CloudFormation stack with an Amazon EC2 resource.
- Tested that the nginx web server ran successfully on the Amazon EC2 instance

